Data Imbalance in Machine Learning

BY
Dr. Adnan Amin
Assistant Professor / Program Director (CS)
School of Computer Science and IT
IMSciences, Peshawar.

Sampling?

- Sampling is the process of selecting a subset of observations from a larger dataset.
- Instead of using the entire dataset, we work with a representative portion to make learning more efficient and practical.
- The goal of sampling is not just reducing data size, but ensuring the sample represents the true underlying distribution.

Why Do We Use Sampling?

- **Computational Efficiency**
  - Large datasets require high memory and processing power
  - Sampling reduces training time and resource usage
- **Handling Imbalanced Data**
  - Adjust class distribution (e.g., reduce majority or increase minority)
  - Helps models learn minority patterns better
- **Faster Experimentation**
  - Enables rapid prototyping and model testing
- **Data Availability Constraints**
  - Sometimes full data is not accessible or too costly to process

Types of Sampling

1. Random Sampling
2. Stratified Sampling
3. Undersampling
4. Oversampling

1. Random Sampling

- Each observation has equal probability of being selected
- Simple and unbiased (if dataset is large enough)

2. Stratified Sampling

- Ensures each class is proportionally represented
- Especially important for imbalanced datasets
- Example:
  - Original: 95% class A, 5% class B
  - Sample maintains same ratio

Undersampling

- Reduces majority class samples
- Goal:
  - Balance dataset by removing excess majority data
- Risk:
  - Loss of important information

Oversampling

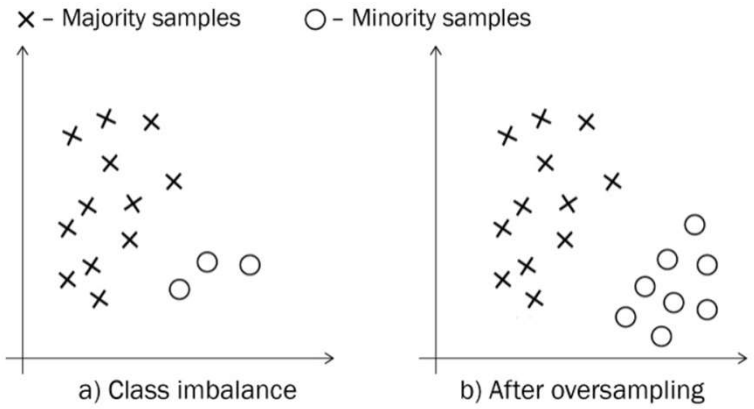
- Increases minority class samples
- Methods:
  - Duplicate samples
  - Generate synthetic data (e.g., SMOTE)
- Risk:
  - Overfitting if not handled properly

Trade-offs in Sampling

Advantage	Disadvantage
Faster training	Possible information loss
Handles imbalance	Risk of bias
Reduced computation	May affect generalization

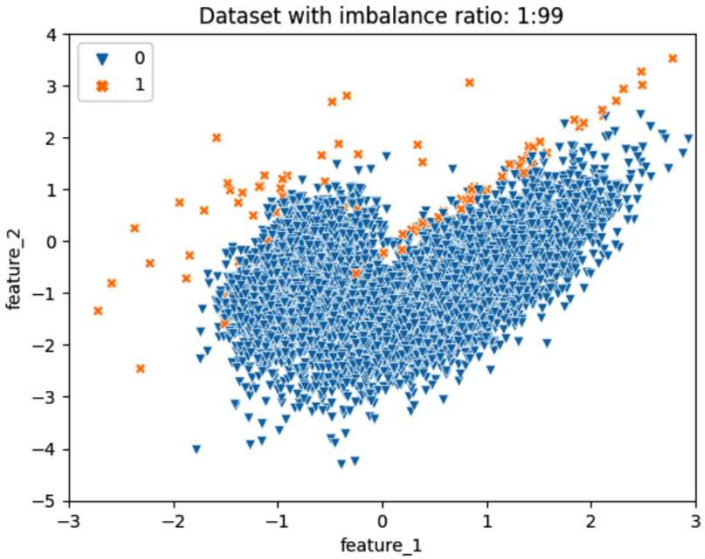
Oversampling

- Oversampling is a sampling technique used to increase the number of instances in the minority class in an imbalanced dataset.
- The goal is to balance the class distribution so that machine learning models can learn patterns from the minority class more effectively.



Why is oversampling needed?

- In imbalanced datasets:
  - Minority class is underrepresented
  - Models tend to ignore it
  - Leads to poor detection (e.g., fraud, disease)
- Oversampling helps by:
  - Giving more importance to minority class
  - Improving recall and detection capability



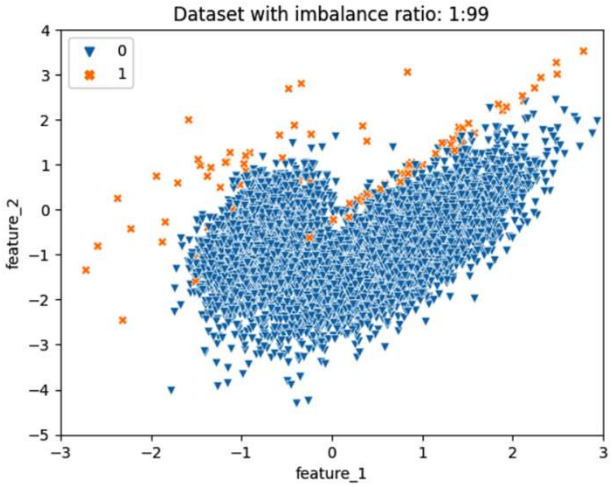
The dataset with an imbalance ratio of 1:99

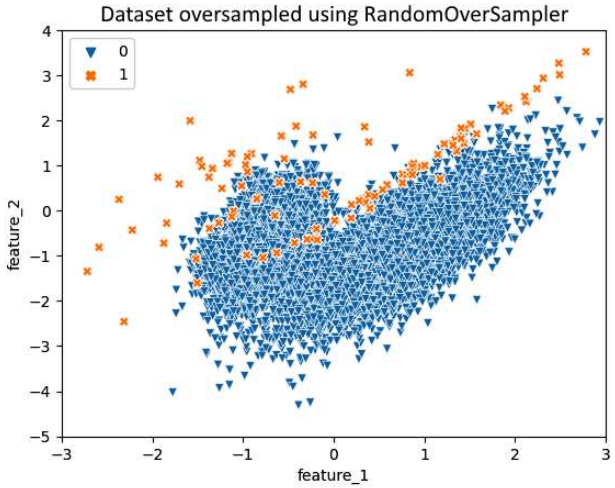
How Does Oversampling Work?

1. Random Oversampling
2. Synthetic Oversampling

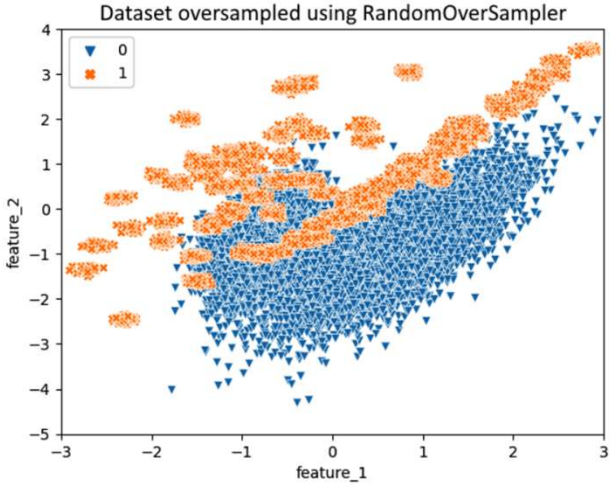
1. Random oversampling

- Randomly duplicates existing minority class samples
- Random oversampling with replacement
  - The simplest strategy to balance the imbalance in a dataset is to randomly choose samples of the minority class and repeat or duplicate them.
- Random oversampling reduces the bias toward the majority class.
- Example:
  - Minority class: 100 samples
  - After oversampling → 500 samples (by duplication)
- Risk: Overfitting (model memorizes duplicates)





Dataset oversampled using RandomOverSampler
(label 1 examples appear unchanged due to overlap)



Result of applying random oversampling with shrinkage=0.2

Problems with random oversampling

- Often lead to overfitting of the model since the generated synthetic observations get repeated, and the model sees the same observations again and again.
- Shrinkage doesn’t care if the generated synthetic samples overlap with the majority class samples.

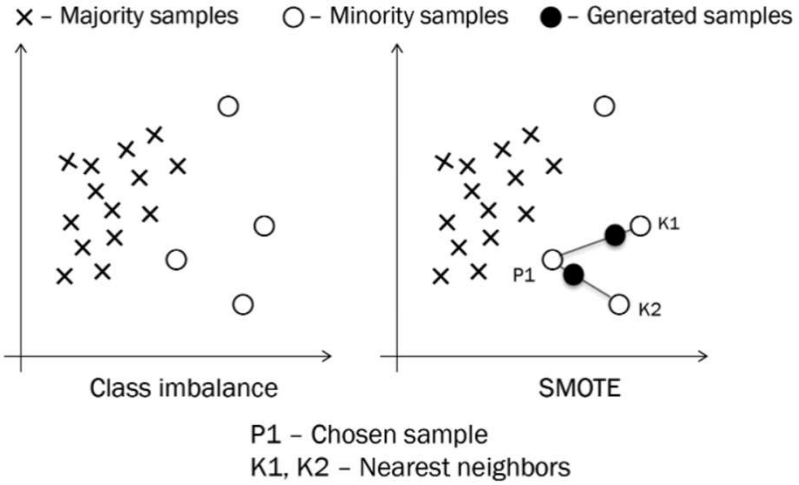
Synthetic Oversampling

- Instead of duplicating, new artificial samples are generated
- Create new samples between existing minority points
  - Without overlapping the self-class sample
  - Without overlapping the majority class sample due shrinkage
- For example,
  - Synthetic Minority Oversampling Technique (SMOTE) & SMOTE variants
  - Adaptive Synthetic Sampling (ADASYN)

SMOTE [1]

- Solves duplication by generating synthetic samples using interpolation.
- Interpolation is playing the role just like reproduction in genetic algorithm (biological analogy).
- Reproduction:
  - Two parents → offspring with mixed characteristics
  - In SMOTE:
    - Two data points → new synthetic point between them.
    - we pick two observations from the dataset and create a new observation by choosing a random point on the line joining the two selected points.

How SMOTE works



Mathematical Representation

A new synthetic sample is generated as:

$$x_{new} = x_i + \lambda(x_{zi} - x_i)$$

where:

- x_i: minority sample
- x_{zi}: one of its neighbors
- $\lambda \in [0, 1]$: random value

- The synthetic samples are generated by taking a random point on the line joining a minority sample to two nearest neighbor majority class samples (right)

Step-by-Step Process:

1.Select minority class samples only

2.Train K-Nearest Neighbors (KNN) on minority data
- Typical value: $k = 5$

3.For each minority sample:
- Find its k nearest minority neighbors

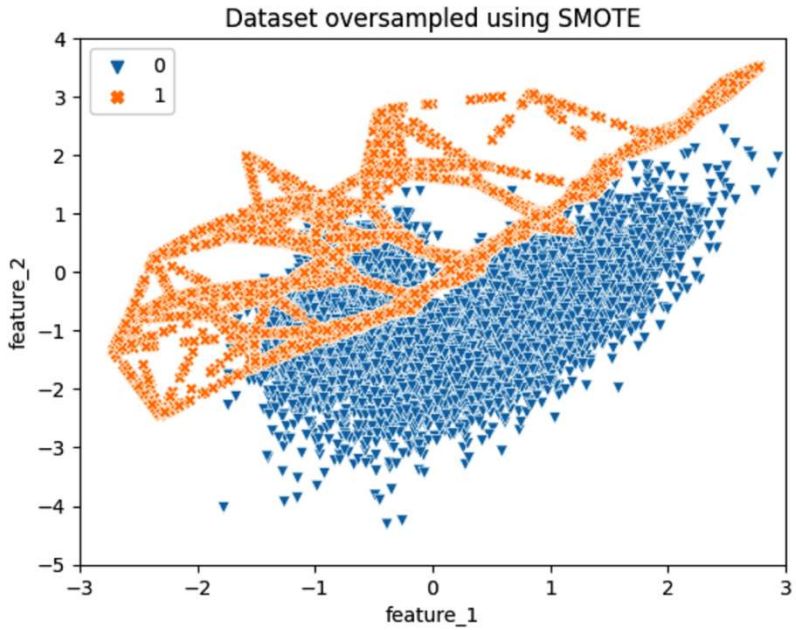
4.For each neighbor:
- Draw a line between the two points

5.Generate a synthetic point:
- Randomly pick a point on the line segment

SMOTE using APIs

- from imblearn.over_sampling import SMOTE
- sm = SMOTE(random_state=0)
- X_res, y_res = sm.fit_resample(X, y)
- print('Resampled dataset shape %s' % Counter(y_res))

- Output:
- Resampled dataset shape Counter({0: 9900, 1: 9900})

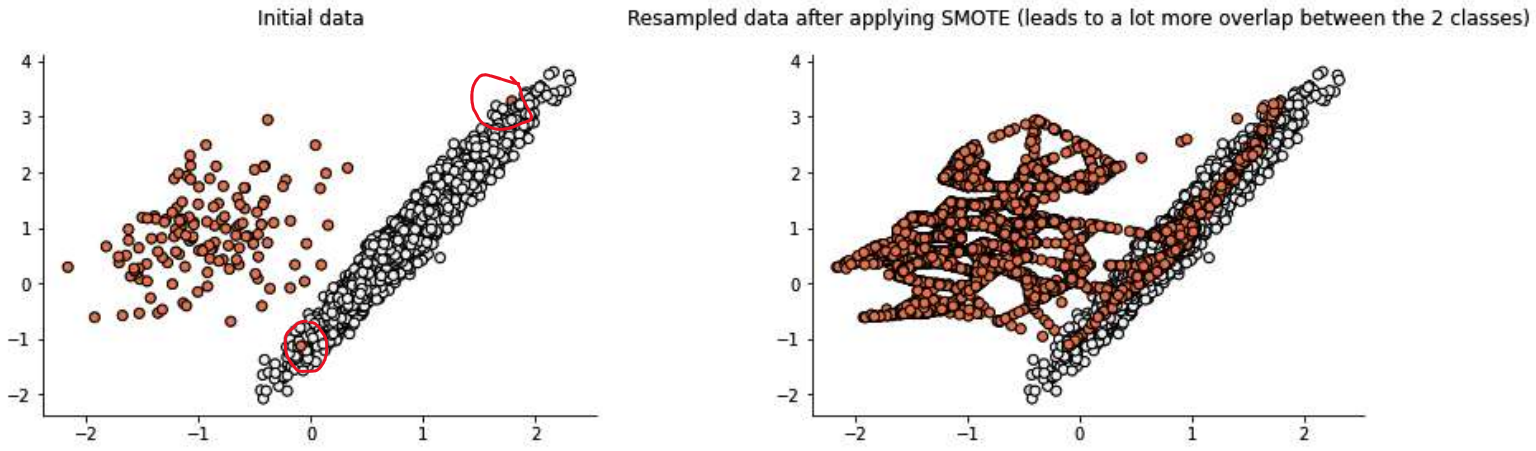


Limitations of SMOTE

- Noise Amplification
- Class Overlap
- Computational Cost

Example

- SMOTE generates minority class samples without considering the majority class distribution, which may increase the overlap between the classes.
- For example, If dataset contains noisy minority samples then SMOTE generates more noise



Binary classification dataset before (left) and after (right) applying SMOTE (see the overlap between two classes on the right)

SMOTE variants

These variants can be applied in certain situation.

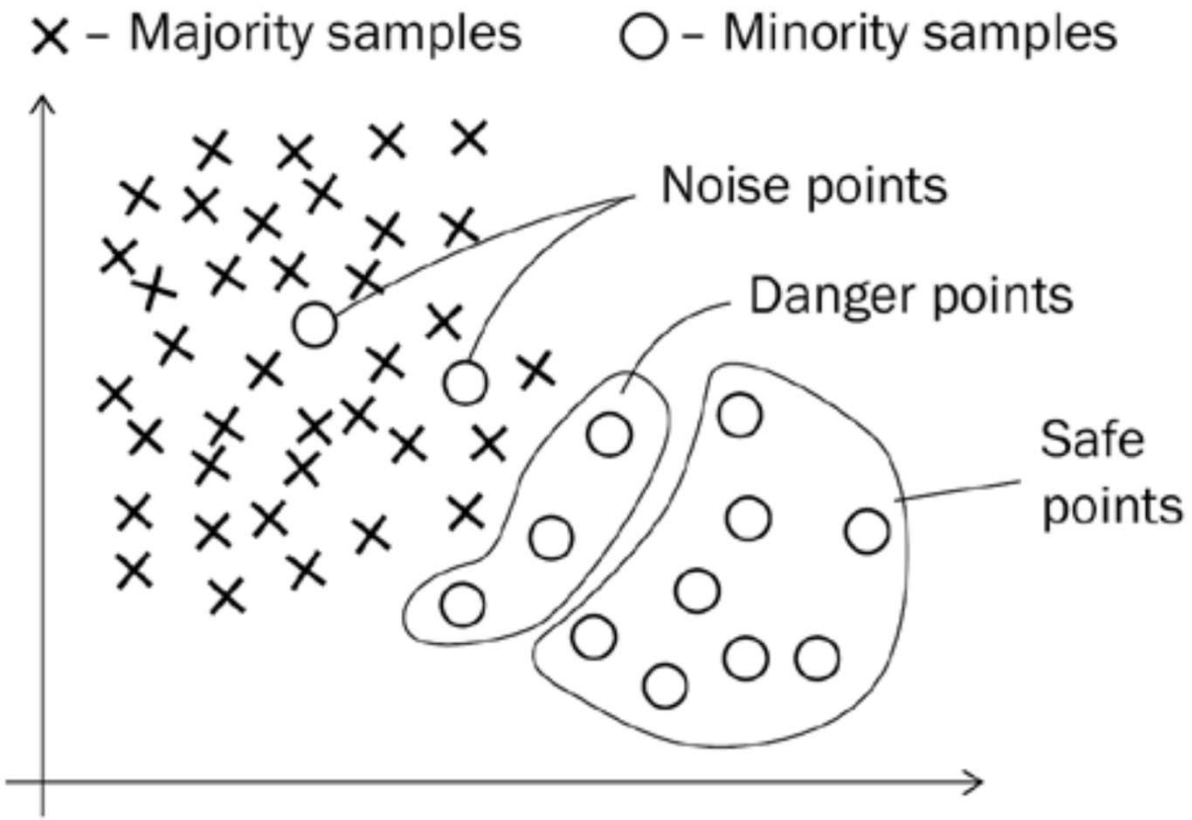
- Borderline-SMOTE
- SMOTE-NC: Categorical Features
- SMOTEN: Categorical Features

Why Do We Need SMOTE Variants?

- While standard SMOTE improves minority representation, it has limitations:
  - Generates samples blindly (ignores class boundary)
  - May increase class overlap
  - Not suitable for categorical data
- SMOTE variants address these specific weaknesses.

Borderline-SMOTE [2]

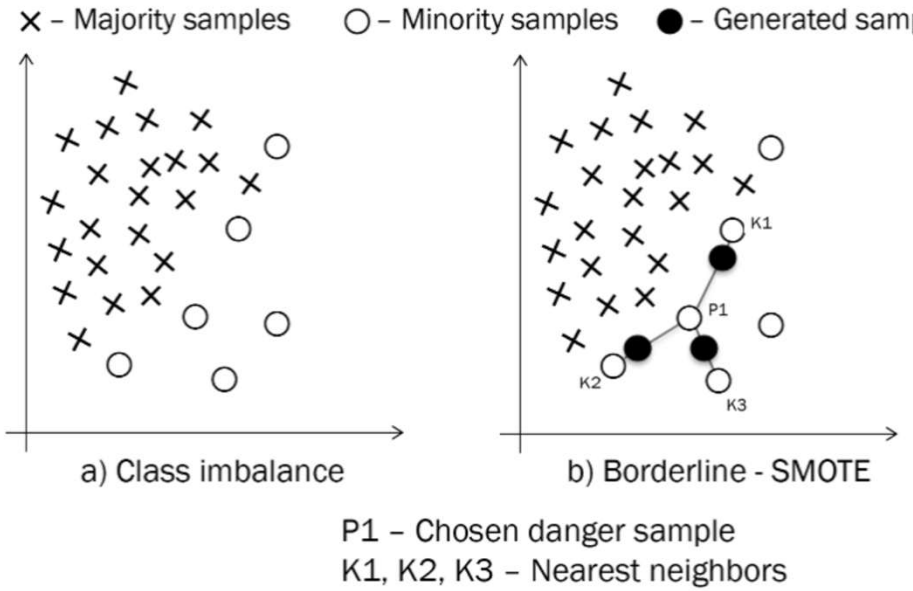
- Focuses only on minority samples near the decision boundary, where misclassification is most likely.
- Why Focus on Boundary Points?
  - The examples near the classification boundary are more prone to misclassification than those far away from the decision boundary.



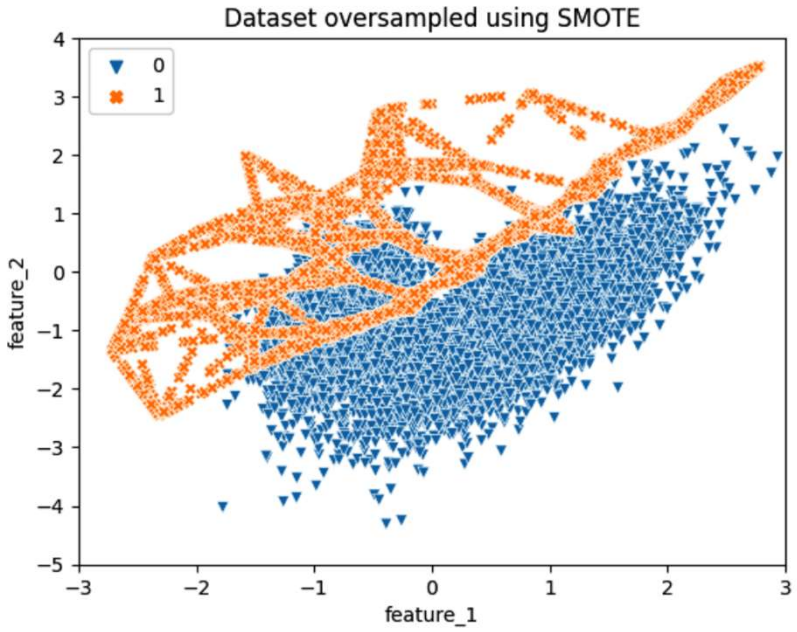
Step-by-step algorithm for Borderline-SMOTE

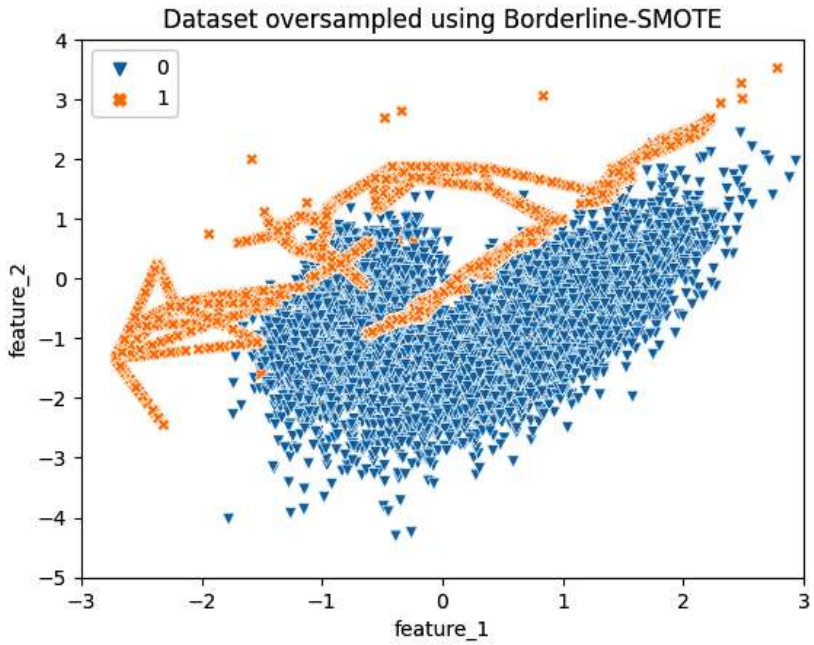
1. We run a KNN algorithm over the whole dataset.
2. Then, we divide the minority class points into three categories:
    1. Noise Points: Surrounded entirely by majority class (outlier)
    2. Safe Points: Mostly surrounded by minority class (ignore)
    3. Danger Points: Surrounded mostly by majority class (Located near decision boundary)
3. Then, we train a KNN model only on the minority class examples.
4. Apply SMOTE only on Danger Points
5. Synthetic samples are generated along neighbors, and focus is strictly on boundary region

Illustrating Borderline-SMOTE



- Plots of majority and minority class samples
- Synthetic samples generated using neighbors near the classification boundary


Dataset oversampled using SMOTE
0
1
feature_2
feature_1
4
3
2
1
0
−1
−2
−3
−4
−5
−3
−2
−1
0
1
2
3


Dataset oversampled using Borderline-SMOTE
0
1
feature_2
feature_1
4
3
2
1
0
−1
−2
−3
−4
−5
−3
−2
−1
0
1
2
3

Key finding

- Borderline-SMOTE oversampling, which resulted in a 4%-7% increase in PR-AUC but increased the training time by 25%-35%.
- A hybrid of random oversampling and random undersampling, where they randomly oversampled the minority class and undersampled the majority class. It led to a 6%-10% improvement in PR-AUC and increased the training time by up to 25%.

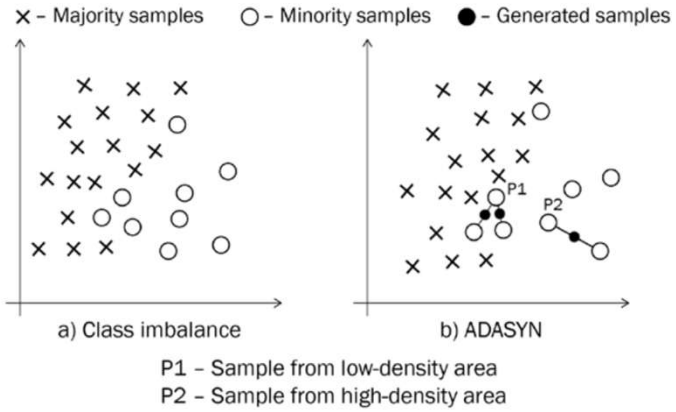
Adaptive Synthetic Sampling (ADASYN)

Key Intuition

- Minority samples near class boundaries or surrounded by majority samples are harder to learn.
- ADASYN prioritizes these samples by generating more synthetic instances around them.
- This results in a bias toward difficult regions, unlike SMOTE's uniform strategy.

ADASYN [3]

- ADASYN focuses on harder-to-classify minority class samples since they are in a low-density area.
  - While SMOTE doesn't distinguish between the density distribution of minority class samples.
- ADASYN assigns a data-driven weight distribution to minority samples based on their classification difficulty.



The hardness of samples based on how many majority observations are its neighbors.

More synthetic data is generated for samples that are harder to classify

Comparison with SMOTE

Aspect	SMOTE	ADASYN
Sampling strategy	Uniform	Adaptive (based on difficulty)
Focus	All minority samples equally	Harder minority samples
Use of majority class	Not explicitly used	Used in KNN to assess difficulty
Goal	Balance dataset	Improve learning in complex regions

Working of ADASYN

- Step 1: Train KNN
  - A K-Nearest Neighbors (KNN) model is trained on the entire dataset.
- Step 2: Compute Hardness Factor
  - For each minority sample, compute its hardness level:

$$r_i = \frac{M_i}{K}$$

  - Where:
    - M_i =number of majority class neighbors
    - K= total number of nearest neighbors
  - Interpretation:
    - $r_i \approx 0$:Easy to classify
    - $r_i \approx 1$:Hard to classify

Working of ADASYN cont…

- Step 3: Generate Synthetic Samples
  - Synthetic samples are generated proportional to r_i .
  - For each minority sample:
    - Draw a line between the sample and its neighbors.
    - Generate new samples along this line.
  - Higher difficulty: more synthetic samples generated

Key Advantages

- Focuses learning on decision boundaries
- Reduces bias toward easy minority samples
- Improves performance in highly imbalanced datasets
- Adaptive and data-driven

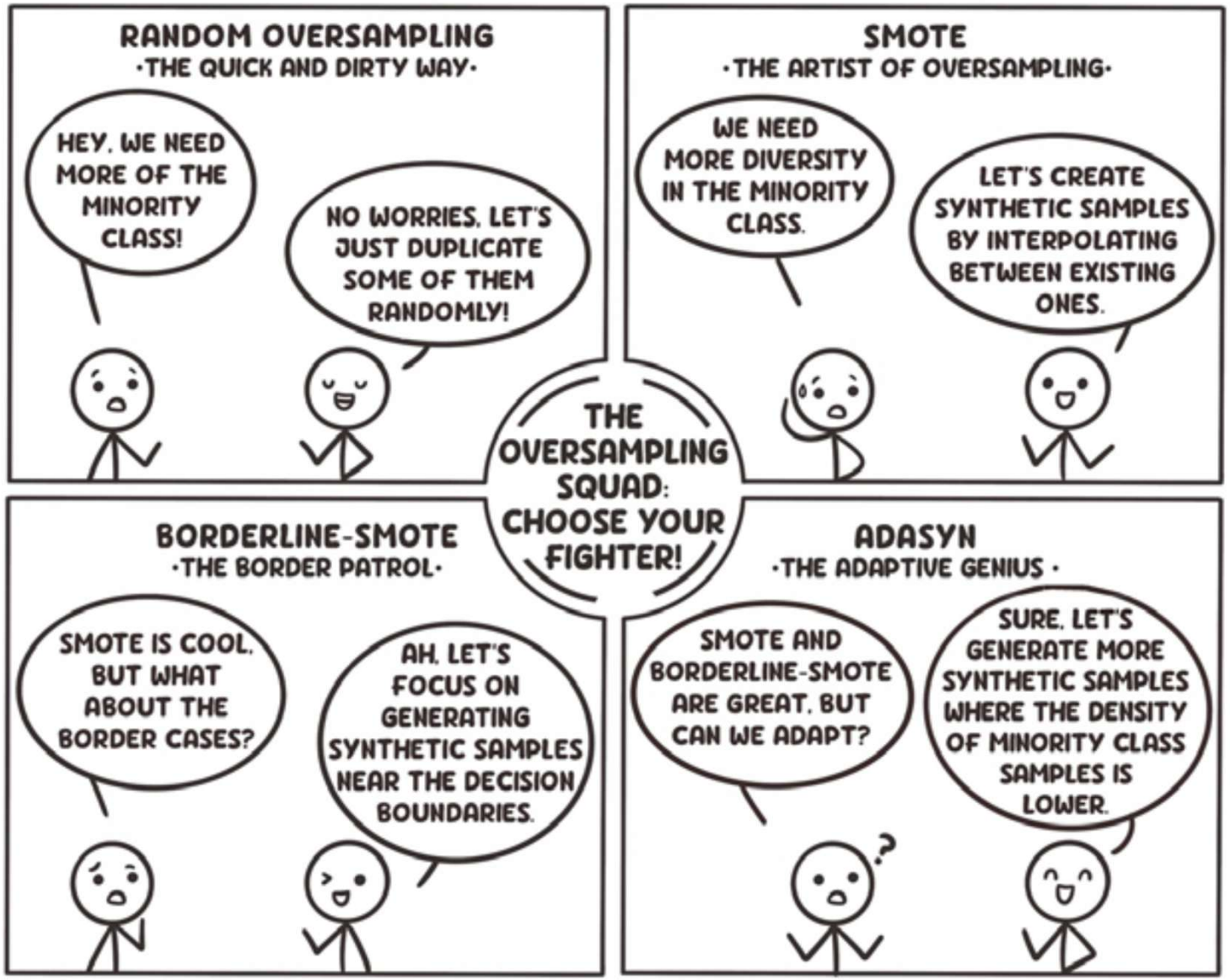
Limitations

- May introduce noise if difficult samples are actually outliers
- Slightly more computationally expensive than SMOTE
- Sensitive to choice of K in KNN

Summary

- ADASYN enhances oversampling by shifting the learning focus toward challenging regions of the feature space, making it particularly effective for real-world datasets where class overlap and imbalance coexist.

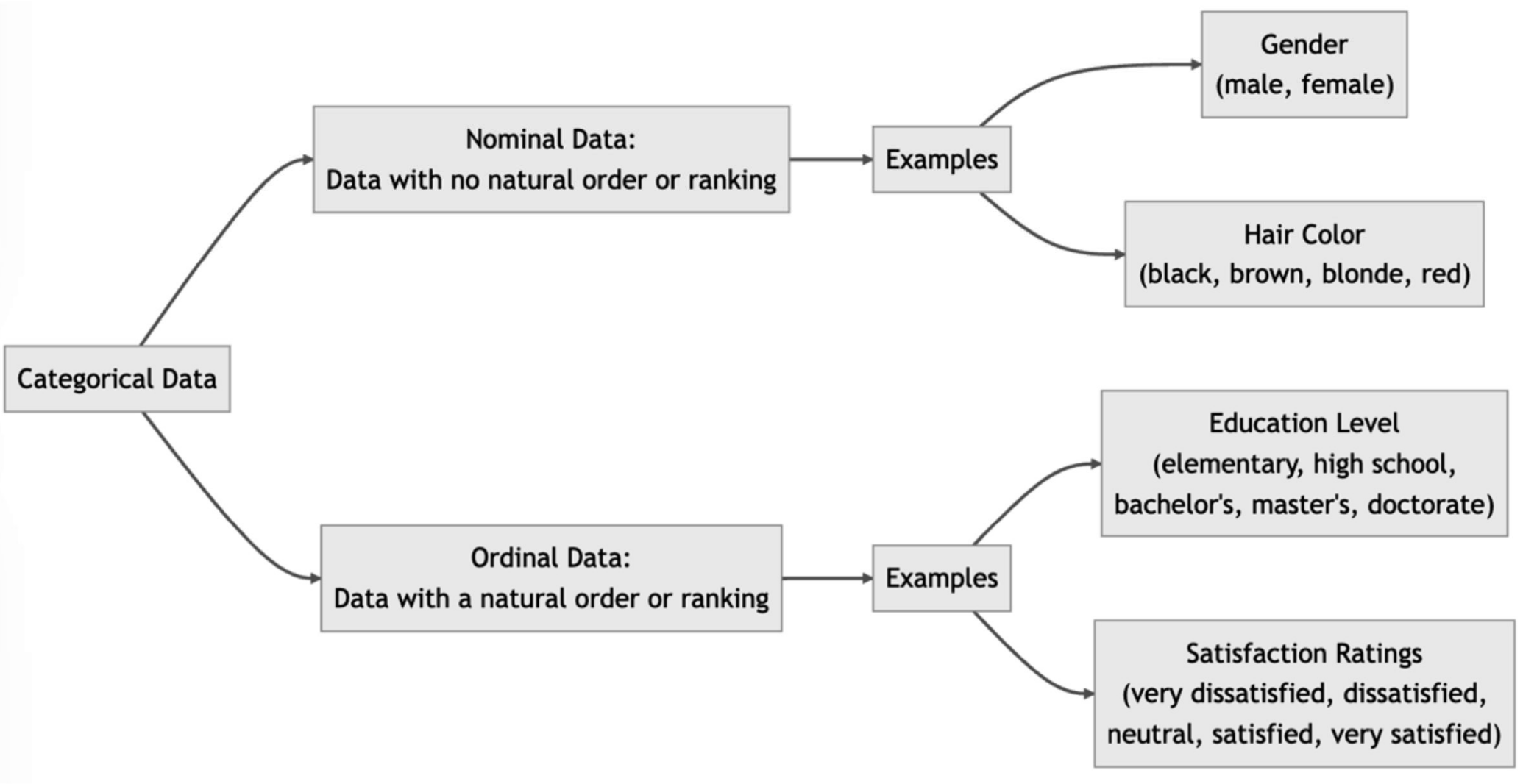
- A memory aid summarizing various oversampling techniques



SMOTE VARIANTS: SMOTE-NC and SMOTEN
Handling Categorical Features in Imbalanced Learning

Categorical Features

- A **categorical feature** takes values from a finite set of discrete categories. These can be divided into:
- **Nominal features**: No inherent order
  (e.g., color, gender, shape)
- **Ordinal features**: Have a natural ordering
  (e.g., low < medium < high)


Categorical Data
Nominal Data:
Data with no natural order or ranking
Examples
Gender
(male, female)
Hair Color
(black, brown, blonde, red)
Ordinal Data:
Data with a natural order or ranking
Examples
Education Level
(elementary, high school,
bachelor's, master's, doctorate)
Satisfaction Ratings
(very dissatisfied, dissatisfied,
neutral, satisfied, very satisfied)

Challenges with SMOTE and Categorical Data

- The standard SMOTE operates by interpolation between minority class samples:
  - It generates synthetic points along the line connecting two samples.
  - This works well for continuous numerical data.
- Problem with categorical data:
  - If categories are encoded numerically (e.g., yes = 1, no = 0), interpolation may produce invalid values:
  - Example: 0.3 is not a valid category
  - Therefore, SMOTE cannot be directly applied to categorical features.

Basic Alternatives

- **Ordinal Encoding:** For ordinal features, we can use:
  - scikit-learn's OrdinalEncoder
  - Maps categories → integers (0, 1, 2, ...)
- **Random Oversampling:** The Random Oversampling method:
  - Simply duplicates minority samples
  - Works with categorical data
  - Does not generate new synthetic values

SMOTE Variants for Categorical Data

- To overcome SMOTE's limitations, specialized variants are introduced:
  - SMOTE-NC (Synthetic Minority Oversampling Technique – Nominal Continuous)
  - SMOTEN (Synthetic Minority Oversampling Technique for Nominal Data)

1. SMOTE-NC

- SMOTE-NC is designed for mixed datasets containing:
  - Numerical (continuous) features
  - Categorical (nominal) features
- How it works:
  - Numerical features → generated using SMOTE interpolation
  - Categorical features → assigned using majority voting among KNN neighbors
- Important:
  - Requires at least one numerical feature
  - Cannot work with purely categorical datasets
- Example Use Case:
  - Dataset: Age (numeric), Gender (categorical), Income (numeric)

2. SMOTEN

- SMOTEN is designed for fully categorical datasets.
- How it works:
  - Treats all features as categorical
  - Generates synthetic samples using:
    - Majority voting from nearest neighbors
  - Uses a specialized distance metric (Value Distance metric VDM)

Value Distance Metric (VDM)

- Value Distance Metric computes similarity between categorical values based on class distribution:
  - Two values are considered similar if:
  - They appear in similar class distributions
- Advantage:
  - Captures relationships between categorical features and labels

Comparison of SMOTE Variants

Method	Supported Data	Strategy
SMOTE	Numerical only	Interpolation
SMOTE-NC	Mixed (numerical + categorical)	Interpolation + majority vote
SMOTEN	Categorical only	Majority vote (VDM-based)

- Use SMOTE → when all features are continuous
- Use SMOTE-NC → when dataset is mixed
- Use SMOTEN → when dataset is fully categorical
- Use Random Oversampling → as a simple baseline

Practical Guidelines for Using Oversampling

- Start with a baseline model (no sampling)
- Apply Random Oversampling as a simple benchmark
- Handle categorical features:
  - Encode → One-hot / Ordinal
  - OR use SMOTE-NC / SMOTEN directly
- Experiment with:
  - SMOTE
  - Borderline-SMOTE
  - ADASYN
- Evaluate using:
  - Precision, Recall, F1-score
  - ROC-AUC / PR-AUC

References

- [1] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, *SMOTE: Synthetic Minority Over-sampling Technique*, jair, vol. 16, pp. 321–357, Jun. 2002, doi: 10.1613/jair.953.
- [2] H. Han, W.-Y. Wang, and B.-H. Mao, *Borderline-SMOTE: A New Over-Sampling Method in Imbalanced Data Sets Learning*, in Advances in Intelligent Computing, D.-S. Huang, X.-P. Zhang, and G.-B. Huang, Eds., in Lecture Notes in Computer Science, vol. 3644. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 878–887. doi: 10.1007/11538059_91.
- [2] Haibo He, Yang Bai, E. A. Garcia, and Shutao Li, *ADASYN: Adaptive synthetic sampling approach for imbalanced learning*, in 2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence), Hong Kong, China: IEEE, Jun. 2008, pp. 1322–1328. doi: 10.1109/IJCNN.2008.4633969.